

Build your personal *learning assistant*.

Choose a study note. Explain the choice.

Build a small product you can inspect and test.

🕒 15-20 min core reading · 30-45 min project practice · 12 short sections

Build something you can use, inspect, and test. Our first project recommends a note from a small catalog, explains the choice, and handles the cases where a useful answer is not available.

WHAT YOU WILL BUILD

A complete path: preferences → eligible notes → ranked choice → an explanation with known and missing facts.

Two ways to run it: a working browser demo and a downloadable Python project. Both start with explicit rules; Python can optionally add an LLM-written draft using Chapter 10's adapter.

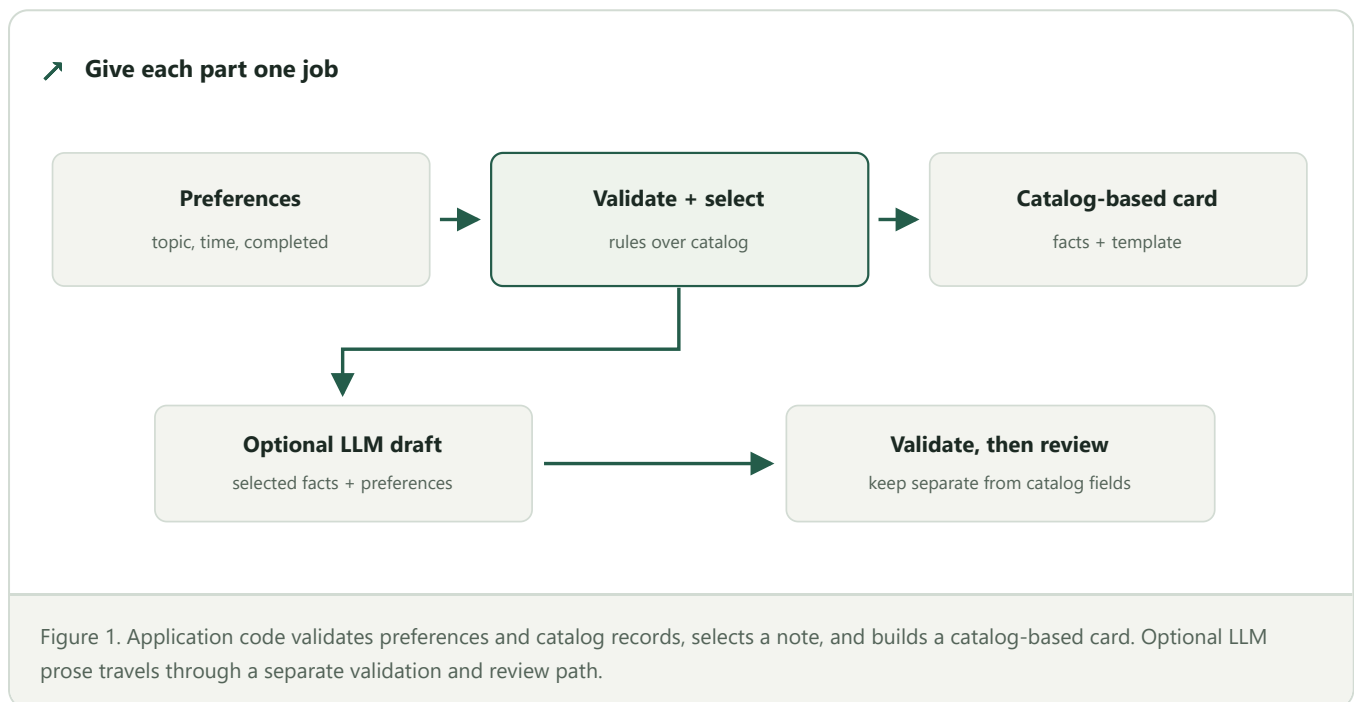
Reading path: follow the core sections, try the demo, then run the project. Expand the deeper notes when you want more detail.

1. Start with a small product promise

“Help me choose a note about how models learn, estimated at no more than 30 minutes. Exclude notes I have completed. I prefer an introductory explanation.” This request gives us a task small enough to build and test.

The application returns one suggested note, the catalog facts that explain the choice, and exam timing if supplied. If nothing qualifies, it says so and lets the learner change the request. It never promises that the learner will benefit from the note.

An **acceptance criterion** is an observable condition for calling a feature successful. Here, the chosen note must exist in the catalog, match the required topic, have an estimate within the time limit, and be absent from the learner’s completed list. A missing exam date must stay unknown. These conditions are more useful than “make the AI good.”



The first working version is a **baseline**: a simple reference implementation to compare improvements against. Our selection rules and template explanation need no trained model. Adding an LLM makes the explanation stage generative; it does not turn the ranking rule into learned personalization. This project connects Chapters 4-5’s software foundations with Chapters 9-10’s prompting and validation.

2. Keep catalog facts separate from preferences

The downloadable study pack has seven authored notes. Each record combines a stable ID, title, text, topic tags, level, study-time estimate, and optional exam metadata for that topic. N01, N02, and N03 retain Chapter 5’s titles, text, and estimates. For this compact project, timetable metadata is bundled with each note; a larger app would keep separately versioned timetable records. A **stable ID** identifies the same record even when its display title changes. For example:

```
{
  "id": "N01",
  "title": "Overfitting",
  "topics": ["learning", "data"],
  "level": "introductory",
  "estimated_minutes": 20,
  "exam_date": null,
  "text": "A model can fit training examples too closely and perform poorly on new cases."
}
```

Preferences are a different object: `topic`, `max_minutes`, `prefer_introductory`, and `completed_ids`. The learner can change them without rewriting the note's study-time estimate. `null` means a value is unknown; it does not mean zero minutes or that no exam exists. A study estimate is planning metadata, not measured completion time. Marking a note completed is a learner input, not proof of mastery.

The catalog's `version` identifies this input snapshot. Recording it alongside the request helps explain why a result changed after an edit. The topic names are real; level labels, study estimates, and exam dates are authored teaching metadata. They are not current course or timetable information.

▼ Go deeper: data validation is not factual verification

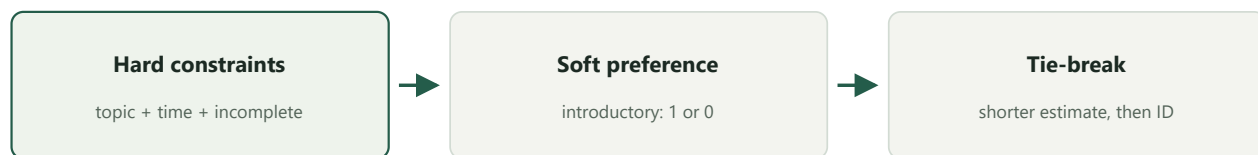
The project rejects duplicate note IDs, unknown topic tags, unexpected fields, and invalid types. It accepts a null study-time estimate as missing data. These checks make records usable by the program; they do not prove that a real-world note actually has that study-time estimate. A real catalog also needs provenance, update dates, course-specific timetable information, and a process for correcting errors. We will develop data-quality concepts in Chapter 12.

3. Filter hard constraints before ranking

A **hard constraint** must hold for a result to qualify. Our hard constraints are the required topic, maximum study-time estimate, and exclusion of completed notes. The "any" topic option removes only the topic restriction. "Maximum 30 minutes" includes a note estimated at exactly 30 minutes.

A **soft preference** helps choose among qualifying results. Here, an introductory level is preferred, not required. An advanced note can still be returned if it is the best eligible option under our policy.

➤ **Filter first. Then compare eligible notes.**



A 45-minute estimate exceeds a 30-minute budget.

An unknown study-time estimate cannot establish that the limit is satisfied either.

Figure 2. Hard constraints decide eligibility. The soft introductory-level preference only orders eligible notes; it cannot rescue a note that breaks a required constraint.

Overfitting passes the default request: it has the learning tag, is estimated at 20 minutes, and is not yet completed. Embeddings is estimated at 45 minutes and has only the search tag, so it fails two constraints. Backpropagation has a learning tag but no study-time estimate; the program cannot establish that it fits the time limit, so it is excluded too.

▼ **Go deeper: requirements should match the wording**

“I would prefer an introductory note” allows a tradeoff. “Only show introductory notes” expresses a hard constraint and would need a separate filter. Likewise, “under 30 minutes” strictly means less than 30, while this project’s maximum-time control means at most 30. Translate wording into explicit rules before implementation. If an interface quietly changes the meaning, even perfectly executed code can solve the wrong problem.

4. Rank candidates with a transparent rule

A **candidate** is a note still under consideration. **Ranking** puts candidates in a preferred order. Our small baseline assigns an eligible note one point if introductory level is preferred and the note is introductory; otherwise it gets zero. Higher scores come first.

Eligible note for the default request	Introductory-level score	Study-time estimate
Overfitting · N01	1	20 minutes
Train-validation splits · N04	1	25 minutes
Gradient descent · N02	0	15 minutes

We break equal scores by shorter study-time estimate, then note ID. Overfitting wins over Train-validation splits because 20 is less than 25. Gradient descent follows both because its score is lower, even though it is shorter than Train-validation splits. This is a deliberate product policy, not a mathematical law about better notes.

The score is neither a probability nor a predicted rating. We have not trained anything or used other learners' behavior. Large recommenders often separate candidate generation, scoring, and re-ranking; this project's filter-and-sort pipeline is a small, explicit design. See [Google's recommendation architecture overview](#).

▼ Go deeper: cold start, ties, and learning later

Cold start means making recommendations with little interaction history for a new learner or item. Asking for preferences is one starting point. Later, ratings, quiz performance, or item features could support learned scoring, but you would need suitable data and evaluation. Even a learned score often needs explicit tie-breaks and product constraints. Stable sorting rules make this baseline reproducible; they do not make it personalized beyond the supplied preferences.

5. Try the complete recommendation flow

Use the demo below before reading the implementation. Start with the defaults. Then mark Overfitting as completed: Train-validation splits becomes first. Turn off the introductory-level preference as well: Gradient descent now wins because its 15-minute study-time estimate is shorter than the 25-minute estimate for Train-validation splits. Finally, lower the learning time limit to 10: nothing qualifies.

YOUR FIRST COMPLETE PROJECT

Local rules · authored study pack

What should I study next?

Choose your preferences and inspect the recommendation. Everything here runs in your browser; the explanation uses a template.

Default run: learning, no more than 30 minutes, prefer an introductory level, no completed notes. Overfitting has an estimated study time of 20 minutes, within your 30-minute limit. Topic: learning, data. Level: introductory. Selected by the stated ranking rule; learning benefit is not guaranteed. Exam timing remains unknown.

Note	Decision	Score
Overfitting	Eligible	1
Embeddings	Topic does not match; Over time limit	n/a
Train-validation splits	Eligible	1
Gradient descent	Eligible	0
Features and labels	Topic does not match	n/a
Matrix multiplication	Topic does not match	n/a
Backpropagation	Study-time estimate unknown	n/a

Ranking: N01 → N04 → N02. At a 10-minute limit, no learning-topic note qualifies. Excluding N01 at 30 minutes selects N04. Turning off the introductory preference in that last case selects N02.

The explanation is assembled from catalog fields and the request. It reports why the selected item qualifies, without inventing learning outcomes or exam timing. The decision table exposes the rule's operation, not a model's private reasoning. Downloading the result saves an inspectable JSON snapshot; changing preferences alone does not save them.

6. Run and understand the Python project

Extract all files into one folder and open a terminal there. Python 3.10 or newer is sufficient; no third-party packages are needed for this workbook.

```
py study_app.py
py evaluate.py
py -m unittest -v test_study_app.py
```

Use your working `python` or `python3` command if `py` is unavailable. The first command prints a result with `status: ok`, note ID N01, and an unknown exam date. The other commands run the prepared evaluation and code tests. None calls a model service.

Open `preferences.json` and put `"N01"` in `completed_ids`; rerun the first command. The recommendation should change to N04. Set `prefer_introductory` to `false`; it should change to N02. These are the same decisions you explored in the browser.

The main `recommend` function validates inputs, creates an exclusion-reason list for each record, sorts eligible notes, and builds a result. Its sorting key is compact:

```
eligible.sort(key=lambda m: (
    -int(p["prefer_introductory"] and m["level"] == "introductory"),
    m["estimated_minutes"],
    m["id"],
))
```

Python compares tuple elements from left to right. Negating the score puts one-point notes before zero-point notes in an ascending sort. Study-time estimate and ID resolve ties. Missing study-time estimates have already been filtered out.

▼ Go deeper: where the browser and Python run

The handbook demo executes JavaScript locally; it does not send requests to the downloaded Python program. The workbook implements the same rules in Python, and automated checks compare their outputs. In a deployed application, a browser could send preferences to a server that owns validation, catalog access, and model credentials. You would then need an HTTP contract, loading and error states, and server-side validation. Chapter 5 introduced that boundary; this project keeps the two learning environments independent.

The folder's README maps each file. `study_app.py` owns the project flow, while `llm_core.py`, `provider.py`, and `errors.py` reuse Chapter 10's service boundary. Use `study_app.py` as the entry point.

7. Add an LLM explanation as a separate stage

An LLM can phrase the explanation more flexibly. It should receive the learner preferences and the selected note's facts, rather than being asked to invent an entire catalog. In this project, S1 holds preferences, S2 holds the selected note, and S3 holds its supplied timetable information. N01 is a note ID; S2 is a source ID within the current prompt.

First exercise the integration offline:

```
py study_app.py --draft template
py study_app.py --draft unsupported
```

The template mode constructs an authored response and sends it through Chapter 10's real envelope, JSON, field, and source-ID checks. It is not generated by an LLM. Unsupported mode deliberately inserts an unsupported 2026-12-01 exam date; it demonstrates that a false fact can pass structural checks.

The result keeps two separate objects. `recommendation` is the catalog-based card. `draft.candidate` contains the optional prose and the familiar four Chapter 10 fields. Its `review_required` status means a person must compare claims with the supplied records. The draft never overwrites the card's note or exam-date field. Valid structure still allows factual mistakes, as described in [OpenAI's Structured Outputs guidance](#).

▼ Go deeper: optional live mode and the review boundary

With the local credentials and compatible model configuration described in Chapter 10, explicitly run `py study_app.py --draft live`. This transmits the selected records and preferences and can incur API charges. Keep credentials outside the handbook browser and source files. The download README explains setup and limitations; no live API call was made for this edition.

The project makes at most one provider call for a requested draft and none when no note qualifies. The browser demo remains offline. Check the draft's selected title, study-time estimate, preference claims, source support, and exam timing before using its wording. Separating the draft limits its ability to corrupt catalog fields, but does not make its prose trustworthy. Prompt instructions alone are not evidence verification.

8. Make unhelpful outcomes understandable

An empty result is sometimes the correct answer. With learning and a 10-minute maximum, "No matching note" respects the learner's request. Quietly increasing the limit would conceal a product decision the learner did not make.

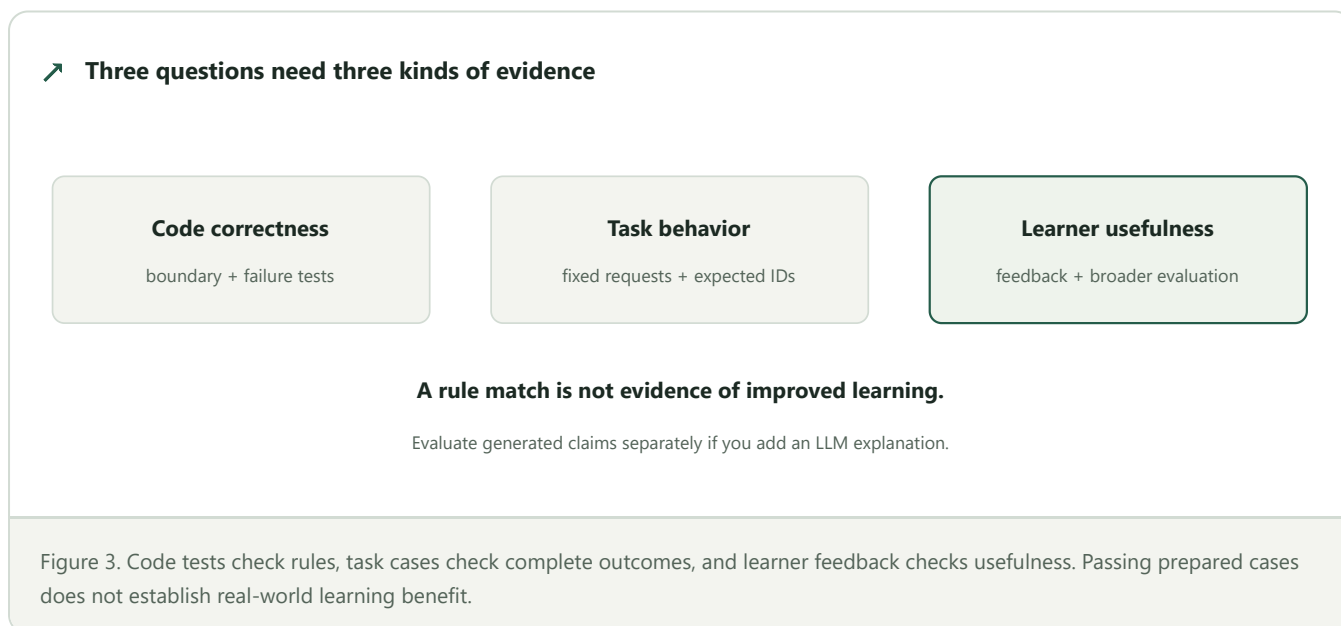
Situation	Project behavior
Invalid preference types or unknown IDs	Stop and describe the input problem
Valid request, no eligible note	Return <code>no_match</code> ; suggest changing preferences
Exam date missing	Keep <code>null</code> ; show “unknown in this catalog”
Optional provider fails	Keep the catalog-based card; record the draft failure
Draft passes structure checks	Keep it separate with <code>review_required</code>

Keeping a usable simpler result when an optional service fails is **graceful degradation**. The interface must distinguish that result from a successful model response. In the CLI, a normal result or no match exits 0; a requested draft failure exits 1; invalid input or setup exits 2. A successful exit code describes program behavior, not factual correctness of model prose.

9. Evaluate the product, not just the happy path

The workbook’s `eval_cases.json` contains 12 requests and expected note IDs, including expected no-match results. These expectations were written for the stated policy; the recommender does not generate its own answer key. Running `evaluate.py` compares actual IDs with that file and reports each case.

The default catalog passes all 12 cases. That means **12 prepared policy checks passed**, not “100% recommendation accuracy.” Cases cover exact time boundaries, completion exclusions, soft-preference changes, topic tags, and empty results. They do not measure whether learners improve after studying the selected notes or whether a live LLM writes accurate explanations.



Code tests add different checks: every eligible note respects the constraints across many time limits; duplicate IDs fail; missing study-time estimates are excluded; ties are stable; provider failures preserve the card; and no-match requests make zero provider calls. The tests use Python’s standard `unittest` framework.

▼ Go deeper: evaluating an improvement fairly

To compare an LLM explanation with the template, keep the same requests and catalog. Check factual support, correct handling of missing data, clarity, latency, and cost; collect learner feedback for usefulness. Count unsupported claims separately from valid-JSON rates. A nicer sentence is not an improvement if it invents an exam date. These prepared cases are development checks. If you tune prompts or a learned ranker against them, preserve separate unseen evaluation cases for a less biased final assessment. Real timetables can change by course and term.

10. Practice: predict, change, and inspect

Exercise 1: separate a rule from a preference

The learner requests learning, at most 15 minutes, prefers introductory level, and has completed N01. Which note wins? Does its level disqualify it?

▼ Reveal the result

N02, Gradient descent, wins. N01 is completed, N04 is estimated at 25 minutes, and N07 has no known study-time estimate. N02 is a not-yet-completed learning note estimated at 15 minutes, so it meets the hard constraints. Its advanced level gives it zero preference points but does not exclude it.

Exercise 2: expose an unsupported claim

Run `py study_app.py --draft unsupported`. Compare `recommendation.exam_date` with `draft.candidate.exam_date`. Which value should a catalog-based “Exam date” field display, and why?

▼ Reveal the distinction

The card must display unknown because its catalog field is null. The draft’s 2026-12-01 claim is unsupported even though the JSON and source IDs passed checks. Keep the draft pending review; do not silently copy it into the card.

Exercise 3: prove the evaluation can fail

Back up `eval_cases.json`, then change the default case’s expected ID from N01 to N05. Run `py evaluate.py`, inspect the mismatch, and restore the original expectation. Why is changing the answer key until the score passes a poor evaluation practice?

▼ Reveal the lesson

The modified evaluation reports 11 of 12 cases passing and exits 1. The default case shows expected N05 and actual N01. Expectations should follow the requirements, not be adjusted merely to match implementation output. A deliberate policy change may require updated expectations, with the reason documented.

11. Check your understanding

Answer all five questions, then read the explanations.

01 An introductory note is estimated at 45 minutes; the learner has a 30-minute limit. What happens?

- A. Its introductory-level score can rescue it.
- B. It is excluded before ranking.
- C. The app silently raises the time limit.

Answer: B. A required constraint decides eligibility. A soft preference only orders notes that already qualify.

02 A note has no study-time estimate in the catalog. Can it pass a maximum-time constraint?

- A. No. The app cannot establish that it fits.
- B. Yes. Missing study-time estimate means zero minutes.
- C. Yes, if the title sounds short.

Answer: A. Unknown is a distinct data state. This project excludes the note and records the reason instead of guessing its length.

03 Two eligible notes have the same introductory-level score. Which rule applies next?

- A. Whichever was inserted first.
- B. The LLM invents a confidence score.
- C. Shorter study-time estimate, then stable note ID.

Answer: C. An explicit tie-break makes repeated runs reproducible. This is a chosen product rule, not a learned measure of learning benefit.

04 An optional LLM draft invents an exam on 2026-12-01, but the source exam date is unknown. What should the UI do?

- A. Overwrite the catalog with 2026-12-01.
- B. Keep the catalog field unknown and flag the draft for evidence review.
- C. Treat valid JSON as verified exam timing.

Answer: B. The catalog-based card and model-written draft are separate. Structural validation does not prove the generated claim.

05 All 12 prepared evaluation cases pass. What does this establish?

- A. Every learner will benefit from the recommendations.
- B. A personalized ML model has been trained.
- C. The implementation matches the expected outcomes for those cases.

Answer: C. These are small, authored acceptance cases for a deterministic policy. They do not measure live model quality or real learner satisfaction.

12. Keep the boundaries visible

Validate the inputs. Filter required constraints. Rank eligible choices. Explain from supplied facts. Review model prose separately. Test the cases where the answer is missing or the service fails.

This is the first stage of the personal learning assistant. Note search, source-backed answers, quizzes, prerequisite checks, and revision scheduling are later extensions. The current project checks supplied preferences and estimates; it does not enforce prerequisites or infer weak topics.

You have completed a small product with an inspectable policy, a working interface, a Python implementation, and repeatable checks. You can now explain which behavior comes from data, which comes from application rules, and where an optional model enters the system.

▼ Quick-reference glossary

Term	Meaning in this project
Acceptance criterion	An observable condition the product should satisfy
Baseline	A simple reference implementation for comparison
Hard constraint	A requirement that determines eligibility
Soft preference	A criterion for ordering eligible options
Candidate / ranking	An item under consideration / its ordering against others
Tie-break	A further rule for equally scored items
Cold start	Making recommendations with little interaction history
Catalog version	The input snapshot used for this run
Graceful degradation	Preserving a useful simpler result when an optional part fails
Acceptance case	A prepared request with an expected product outcome

Sources linked above support the architecture, structured-output limits, and testing tools. The catalog, ranking policy, interface, diagrams, and exercises are original learning examples.

Next: Chapter 12 - Preparing data for machine learning. We move from consuming a small clean catalog to inspecting, cleaning, and splitting data without leaking information into evaluation.